

Arhitectura și Implementarea unei Entități Inteligente Tip „Jarvis” utilizând Ecosistemul Google Antigravity

Apariția inteligenței artificiale generative a catalizat o schimbare fundamentală de paradigmă în ingineria software, marcând tranziția de la medii de dezvoltare statice la platforme orientate către agenți (agent-first). Construirea unei entități autonome de tip asistent personal, deseori conceptualizată sub denumirea de „Jarvis”, transcende simpla orchestrare a unor apeluri către interfețe de programare (API-uri). Aceasta presupune arhitecturarea unui sistem complex care integrează raționamentul multi-pas, cogniția cu persistență pe termen lung, procesarea vocală în timp real (full-duplex) și interacțiunea profundă cu mediul fizic sau sistemul de operare.

Ecosistemul Google Antigravity reprezintă apogeul actual în materie de infrastructură pentru agenți autonomi. Acest raport exhaustiv disecă arhitectura necesară pentru dezvoltarea unui asistent personal avansat, bazându-se pe o analiză detaliată a componentelor de raționament, a paradigmatelor de procesare vocală (Speech-to-Speech vs. Cascadă), a tehnicilor de combatere a latenței acustice și a protocoalelor de comunicare cu dispozitivele de tip Internet of Things (IoT). Documentul culminează cu un plan operațional structurat (blueprint) conceput special pentru a fi ingerat de un agent Antigravity, delegându-i acestuia sarcina scrierii, configurării și lansării bazei de cod.

1. Fundamentele Ecosistemului Google Antigravity

Google Antigravity nu se rezumă la a fi un simplu asistent de completare a codului; este un centru de comandă conceput pentru a sprijini agenți AI care planifică, execută, observă și își validează propriile acțiuni¹. Dezvoltarea unui asistent de tip daemon permanent necesită o înțelegere profundă a modului în care platforma gestionează autonomia, limitările hardware și securitatea execuției.

1.1. Suprafețele de Interacțiune și Utilitatea lor

Ecosistemul Antigravity se ramifică în mai multe suprafețe distincte de interacțiune, fiecare fiind calibrată pentru un flux de lucru specific, dar partajând același nucleu operațional (Agent Harness) bazat pe modelul Gemini 3.7 Flash³. În mod implicit, Gemini 3.7 Flash este modelul echilibrat optimizat pentru raționament, scriere de cod și utilizare de instrumente³.

Pentru dezvoltarea și administrarea unui asistent personal, se disting următoarele suprafețe:

- **Antigravity IDE și Antigravity 2.0 (Desktop):** Oferă o platformă vizuală echipată cu facilități de monitorizare asincronă, unde utilizatorul poate superviza execuția sarcinilor complexe, vizualiza artefactele generate (planuri de implementare, capturi de ecran, structuri de fișiere) și gestiona sarcini programate de tip cron¹.
- **Antigravity CLI:** O interfață de tip terminal (TUI) centrată pe tastatură, care aduce capacitățile agenților direct în sesiunile shell. Este remarcabilă prin viteza de execuție, capacitatea de a lansa comenzi de pe servere la distanță (via SSH) și gestionarea fluidă a sub-agenților paraleli direct din consolă⁴.

- **Antigravity SDK:** Acesta este pilonul central pentru crearea sistemului „Jarvis”. SDK-ul Python oferă control complet asupra ciclului de viață al agentului, abstrăgând mecanica complexă a comunicării cu backend-ul, dar păstrând controlul absolut asupra modului în care agentul gândește și acționează⁴.

1.2. Arhitectura SDK-ului pe Trei Niveluri

Spre deosebire de un simplu script iterativ, agenții construiți cu Antigravity SDK locuiesc în baza de cod ca blocuri de construcție independente (building blocks) ce execută acțiuni concrete¹⁰. SDK-ul este structurat pe trei straturi abstracte care definesc comportamentul cognitiv și limitele de securitate ale asistentului:

- **Stratul 1 (Agent):** Clasa superioară Agent reprezintă punctul de intrare (high-level, batteries-included). Folosită printr-un manager de context asincron (async with Agent(config) as agent:), aceasta preia automat ciclul de viață, descoperirea binarelor necesare, inițializarea instrumentelor și a conexiunii. Pentru un asistent vocal care trebuie doar să ruleze și să primească flux audio, acesta este deseori singurul strat necesar dezvoltatorului⁸.
- **Stratul 2 (Conversație și Sesiune):** Acest strat introduce clasele de tip stateful, precum Conversation, ChatResponse, Step și ToolCall. Aici se definesc mecanismele prin care agentul păstrează istoricul turnurilor de dialog și se ancorează în context. Mai important, acest strat conține motorul de politici de siguranță și funcțiile de tip „hook”. Hook-urile pot fi de trei tipuri: *Inspect* (pentru observare și telemetrie read-only), *Decide* (blocking read-only, prin care se poate bloca o acțiune a agentului) și *Transform* (modificare de date în tranzit pentru sanitizare)⁸.
- **Stratul 3 (Conexiune și Adaptor):** Acest strat de nivel inferior definește modul în care SDK-ul comunică fizic cu motorul modelului. Utilizează instanțe precum LocalConnection pentru o strategie de execuție exclusiv locală sau strategii de conexiune la distanță care permit instalarea agentului pe infrastructura cloud Google, fără modificări structurale ale codului⁸.

1.3. Mecanismele de Siguranță și Instrumentele Implicite

Securitatea by-design este un principiu fundamental. Prin configurația implicită Agent(config), sistemul operează într-un mod de tip read-only, interzicând alterarea fișierelor de sistem sau rularea comenzilor distructive⁸. Pentru ca Jarvis să devină util, clasa LocalAgentConfig expune activarea instrumentelor implicite de sistem. Chiar și cu acestea activate, politica de siguranță încorporată (ex. confirm_run_command()) blochează execuția comenzilor de shell în așteptarea aprobării utilizatorului, o barieră care trebuie gestionată atent prin configurarea CapabilitiesConfig() atunci când se dorește autonomia completă⁸.

Prin trecerea parametrului de configurare pentru generare (GenerationConfig), se poate ajusta profunzimea nivelului de gândire al modelului (MINIMAL, LOW, MEDIUM, HIGH), controlând echilibrul dintre viteză (vitală pentru interacțiunile vocale) și complexitatea rezolvării problemelor logice¹¹.

2. Cognația și Memoria: Sisteme de Persistență

Temporală

Un agent AI suferă de o deficiență critică inerentă arhitecturilor transformer: amnezia la fiecare reinițializare a ferestrei de context. Pentru a construi un asistent permanent, trebuie conceput un subsistem de memorie care rezolvă problema pierderii datelor pe de o parte și degradarea performanței din cauza încărcării excesive a contextului (care atinge limite de siguranță recomandate, de exemplu, la 185.000 tokeni) pe de altă parte¹².

2.1. Paradigma Memoriei Segmentate (Fișiere Mici și RECAP-uri)

Conform experiențelor de implementare în producție, includerea întregului istoric în promptul de sistem transformă agentul într-o entitate lentă și halucinantă. Arhitectura optimă dictează menținerea unor fișiere de memorie extrem de reduse ca dimensiune. Un fișier JARVIS_MEMORY.md nu ar trebui să depășească 3 KB, conținând exclusiv starea imediată a mediului și a sarcinilor curente¹².

Pentru retenția evenimentelor istorice, se instituie un flux de lucru automatizat, apelat, de regulă, zilnic, care scrie loguri cronologice de tip „RECAP”. Agentul înregistrează proactiv acțiunile efectuate (ex. „12h04: Am actualizat configurația de rețea IoT”) într-un fișier separat aferent sesiunii curente. Această structură gestionează în mod silențios trei mari cazuri de utilizare:

1. **Start de zi nouă:** Lipsa unui fișier curent determină agentul să înceapă cu un context curat.
2. **Reluare de sesiune (Mid-Session Resume):** Dacă există un marcaj clar în fișier, agentul deduce că utilizatorul s-a întors după o pauză.
3. **Recuperare după prăbușire (Crash Recovery):** Existența unui RECAP neîncheiat indică faptul că sistemul s-a oprit forțat, iar agentul poate relua exact ultima acțiune notată¹².

2.2. Integrarea Identității și Declanșarea Fluxurilor

Identitatea agentului (instrucțiunile de bază, tonul, permisiunile globale) este plasată într-un fișier permanent, de tipul ~/.gemini/GEMINI.md. Ecosistemul Antigravity încarcă acest fișier automat la orice inițializare a conversației. Această decizie arhitecturală permite inserarea unui mecanism de bootstrapping: agentului i se ordonă prin acest fișier de sistem ca prima acțiune a oricărei noi sesiuni să fie citirea asincronă a logurilor zilnice și rularea procesului de inițializare a memoriei, fără a necesita intervenția sau vreun prompt manual din partea utilizatorului¹².

2.3. Persistența Semantică via SurrealDB MCP

Când datele utilizatorului devin prea complexe pentru loguri Markdown (de exemplu, preferințe personale de-a lungul anilor, arhitectura multor proiecte paralele), abordarea recomandată este integrarea SurrealDB Agent Memory sub forma unui server MCP (Model Context Protocol). Instalarea SurrealDB oferă agentului memorie persistentă capabilă să funcționeze cross-session. Prin editarea fișierului ~/.gemini/config/mcp_config.json, se expun instrumente precum memory_store și memory_recall. Prin mecanismul intern de luare a deciziilor, înainte ca Jarvis să răspundă la o întrebare legată de un proiect, va apela asincron memory_recall utilizând ca argument contextul proiectului (scope-ul), regăsind astfel decizii sau structuri trecute cu o fracțiune din consumul de tokeni al unei ferestre lungi de context¹³.

Tip de Memorie	Suport de Stocare	Utilizare Primară	Timp de Acces
Memoria de Lucru	JARVIS_MEMORY.md	Starea curentă a senzorilor, task-urile din ultimele 2-3 ore.	Instantaneu
Memoria de Sesiune	Jurnale zilnice (RECAP)	Jurnalizarea acțiunilor finalizate pentru crash recovery.	Rapid
Identitate Centrală	GEMINI.md	Comportamentul de bază, tonul conversațional, reguli stricte.	Încărcare la startup
Memorie Semantică	SurrealDB (prin FastMCP)	Relații complexe, date spațiale, contexte cross-project.	Asincron, la cerere

3. Procesarea Vocală: Dezbaterea Arhitecturală

Un asistent cu care nu se poate purta un dialog natural, fluid și fără întârzieri perceptibile reprezintă un simplu panou de control glorificat. Odată cu evoluția inteligenței artificiale, comunitatea de cercetători s-a polarizat între două arhitecturi majore pentru interfețele vocale: modelele „Speech-to-Speech” (Vorbire-către-Vorbire) și pipeline-urile clasice, cunoscute sub numele de arhitectură „în cascadă” (Cascade)¹⁴. Evaluarea precisă a acestora determină fezabilitatea proiectului Jarvis în producție.

3.1. Arhitectura Speech-to-Speech (S2S): Promisiuni și Deficite

Arhitectura S2S utilizează un singur model omnimodal capabil să proceseze nativ tokeni audio (spectrograme) la intrare și să emită direct tokeni audio la ieșire. Din punct de vedere teoretic, S2S aduce avantaje incontestabile legate de latență, înregistrând o reducere cu până la 85% a timpului de răspuns în teste sintetice (scăzând de la o medie de 2000 ms la 200-300 ms). Această performanță este dublată de conservarea „informației paralingvistice”: tonul, ezitățile emoționale, râsetele și ironiile care s-ar pierde prin transcrierea simplă a textului, aspecte esențiale în interacțiunile sociale umane¹⁵.

Totuși, pentru asistenții orientați spre acțiune, care necesită utilizarea fiabilă a instrumentelor software sau decizii precise privind mediul IoT, arhitectura S2S introduce ceea ce analiștii numesc „Problema Controlului”. Într-un sistem S2S, raționamentul, decizia și sinteza acustică

sunt complet fuzionate în interiorul unei singure rețele neurale¹⁷. Erorile sunt tăcute și greu de izolat (fail silently); dacă asistentul răspunde greșit sau nu acționează, este aproape imposibil de identificat dacă a fost o eroare de percepție fonică a modelului sau o halucinație pură de raționament. Din lipsa intermediarului de text, injectarea filtrelor de complianță sau decuplarea logicii de afaceri este extrem de laborioasă, motiv pentru care în 2026 mediile de producție enterprise continuă să evite această rută¹⁷.

3.2. Dominanța Arhitecturii în Cascadă pentru Sisteme Complexe

Arhitectura în cascadă utilizează trei modele distincte, interconectate liniar: Conversia Vorbirii în Text (Speech-to-Text sau STT), Modelul de Limbaj Mare (LLM) pentru raționament, și Conversia Textului în Vorbire (Text-to-Speech sau TTS).

Critica adusă istoric acestui model era „acumularea latențelor”. Într-un model secvențial simplu, se aștepta finalizarea transcrierii, apoi finalizarea raționamentului, apoi finalizarea sintezei, rezultând în timpi morți inacceptabili de 1.5 – 3 secunde¹⁵. Această deficiență a fost invalidată prin adoptarea transmisiunilor asincrone continue (streaming). O cascadă modernă corect arhitecturată permite suprapunerea etapelor: sistemul STT returnează textul parțial în timp ce utilizatorul încă vorbește, iar modulul TTS începe generarea primelor pachete audio imediat ce LLM-ul finalizează primele cuvinte ale răspunsului¹⁵. Timpul de așteptare până la primul byte audio (Time to First Byte - TTFB) a scăzut la intervalul 100-250 ms pentru modelele TTS de generație nouă¹⁶.

Arhitectura în cascadă reprezintă abordarea optimă pentru Jarvis deoarece conferă flexibilitate absolută și transparență. La fiecare graniță dintre componente, se produce un artefact vizibil și înregistrabil (textul). Erorile de rețea, halucinațiile semantice sau degradările vocale pot fi izolate, monitorizate printr-o infrastructură de observabilitate a vocii și corectate individual prin actualizarea unui singur modul¹⁷. Abilitatea agentului Antigravity de a efectua apeluri de funcții (tool calls) este de asemenea incomparabil mai exactă pe baze textuale fundamentate¹⁹.

Metrica Arhitecturală	Arhitectura S2S (End-to-End)	Arhitectura în Cascadă (STT -> LLM -> TTS)
Izolarea erorilor	Opacă, debugging extrem de limitat.	Clară, bazată pe loguri textuale intermediare.
Integrare Tool Calling	Subdezvoltată, imprevizibilă.	Robustă, formatări structurate garantate nativ.
Sensibilitate paralingvistică	Ridicăta (procesează ton, râsete).	Pierdută la granița dintre STT și LLM.
Flexibilitate (Swap Modele)	Zero (vendor lock-in, update greoi).	Totală (orice STT cu orice LLM și TTS dorit).

Implementare Jarvis	Nerecomandată pentru sarcini de sistem.	Arhitectura Optimă de referință.
---------------------	---	---

4. Sistemul Acustic și Fluxul Datelor (Pipeline-ul Audio)

Pentru materializarea arhitecturii în cascadă pe o mașină locală, conservând în același timp un nivel înalt al confidențialității datelor și reducând costurile cu API-urile cloud, selecția instrumentelor de captare, filtrare, transcriere și redare vocale necesită o optimizare matematică a resurselor.

4.1. Recunoașterea Vocală: Silero VAD și Faster-Whisper

Procesarea sunetului captat direct și trimiterea sa constantă către un model de recunoaștere nu este doar scumpă computațional, ci și predispusă la defecte. Modelele de transcriere precum Whisper sunt notorii pentru apariția fenomenelor de buclă infinită pe segmentele lungi de liniște sau halucinaarea unor cuvinte neexistente în mediile cu zgomot static²⁰.

Problema fundamentală constă în segmentarea vorbirii. Tăierea audio-ului în fragmente fixe de lungime, o tehnică rudimentară utilizată anterior, riscă tăierea unei propoziții la jumătate, compromițând complet deducția logică a agentului²¹. Răspunsul la această constrângere este implementarea pre-procesării prin intermediul unei Detectări Neurale a Activității Vocale (Neural Voice Activity Detection - VAD).

Sistemul Silero VAD, o rețea neurală recurentă de nivel enterprise antrenată să distingă cu un grad înalt de acuratețe vocea umană de alte zgomote de fond, este optimul din domeniu. Evaluând blocuri microscopice de sunet (512 eşantioane sau 32 ms la 16 kHz), clasifică fiecare segment în raport cu un prag prestabilit. În dezvoltarea asistentului Jarvis, un parametru de o importanță fundamentală este `min_silence_duration_ms`, setat uzual la aproximativ 500 ms²¹. Prin adăugarea unei toleranțe adiționale pentru suprapunere și umplerea cu zgomot de fond în jurul fragmentelor decupate (`speech_pad_ms`), algoritmul decupează o unitate logică naturală (o propoziție sau o idee) pe baza tiparelor organice de respirație ale omului și trimite pachetul integral către motorul STT²². VAD-urile clasice (ex. WebRTC VAD bazat pe mixturi Gaussiene) suferă din cauza ratelor ridicate de fals-pozitiv în mediile ușor zgomotoase și sunt evitate în această configurație²³.

Transcrierea propriu-zisă folosește nucleul CTranslate2 prin intermediul bibliotecii faster-whisper. Acest pachet elimină necesitatea de procesare cloud și oferă multiple optimizări. Prin instruirea modelului folosind cuantizare INT8, un model large-v3 ocupă sub 3GB de VRAM, funcționând la performanțe aproape identice, din punct de vedere al ratei erorii de cuvânt (Word Error Rate), cu versiunile bazate pe FP16 masive (care consumă 6GB+) pe surse audio clare²².

Configurație Whisper Model	Dimensiune VRAM (aprox.)	Viteza Relativă / Recomandare
----------------------------	--------------------------	-------------------------------

Whisper small (244M)	~500 MB	Dispozitive foarte restrictive. Nu captează perfect contexte specifice.
Whisper large-v3-turbo (INT8)	~3 GB	Echilibrul optim pentru agenți de voce. Latență excelentă.
distil-large-v3	Sub 3 GB	De 6x mai rapid, excelență exclusiv pentru limba engleză.
Whisper large-v3 (FP16)	~6 GB	Precizie absolută, utilizare crescută a resurselor.

4.2. Sinteza Vocală cu Kokoro-82M

Etapă de ieșire din pipeline implică transformarea textului generat de LLM înapoi în unde acustice. În mod istoric, un model cu o acuratețe și intonație bune presupunea o rețea masivă. Totuși, arhitectura s-a îndreptat recent către optimizarea radicală.

Modelul Kokoro-82M, care înglobează doar 82 de milioane de parametri (ocupând sub 100MB spațiu fizic de memorie), oferă performanțe uluitoare, capabile să rivalizeze cu serviciile cloud cu plată, pe dispozitive care nici măcar nu dispun de accelerator GPU, utilizând motorul ONNX Runtime pentru execuția directă pe procesoare (CPU)²⁶.

Arhitectura sa ușoară îi permite să genereze fluxuri (streams) de sunet pe măsura ce primește bucăți din răspuns. Sistemul furnizează o frecvență de 24kHz pentru semnalul acustic de ieșire²⁸. Prin decuplarea procesului LLM, care furnizează cuvintele asincron, de motorul Kokoro care le transformă în fono-secvențe, latența de percepție scade vertiginos, transformând conversația într-o experiență naturală. Astfel se elimină conceptul de „așteptare la procesare”²⁹.

5. Controlul Interactiv: Duplex Complet și Gestiunea Întreruperilor

Calitatea de a fi viu a unui agent asistent nu rezidă în cât de inteligente sunt frazele acestuia, ci în capacitatea lui de a asculta, inclusiv în momentul în care el însuși vorbește. Într-o conversație umană naturală, persoanele se pot întrerupe reciproc („barged-in”). Absența acestui mecanism limitează AI-ul la funcționalitatea unui aparat de tip walkie-talkie (half-duplex)²⁴.

5.1. Problema Fizică a Suprapunerii Audio (Double-Talk)

Când sistemul Jarvis rulează într-o cameră, vocea sintetizată emisă de boxele sale este reflectată de pereți și captată instantaneu înapoi de propriul microfon. Soluția naivă ar duce la o disonanță fatală: algoritmul VAD ar recunoaște vocea agentului drept prezența umană, activând

un nou flux de recunoaștere STT care ar transcrie discursul propriei mașini, inserând-o ca nouă întrebare de la utilizator, forțând agentul într-o buclă infinită în care conversează cu sine însuși²⁰. Mai grav, orice zgomot minor ambiental ar putea declanșa acest fenomen, de unde izvorăște „Paradoxul Reducerii Zgomotului”²⁵.

5.2. Tehnologia de Anulare a Ecoului Acustic (AEC)

Răspunsul ingineresc la acest bruiaj este Anularea Ecoului Acustic (Acoustic Echo Cancellation - AEC). Sistemul AEC preia semnalul audio curat exact înainte să fie trimis spre difuzoare („semnalul de referință”) și antrenează un filtru adaptiv bazat pe profilul camerei (reverberații, distanțe). Ulterior, acest ecou prezis este extras, matematic, din semnalul venit de la microfon³². Situația devine extrem de fragilă în secunda de double-talk, când omul decide să vorbească exact în timp ce agentul generează fraze. Dacă algoritmul continuă să își adapteze filtrul în double-talk, el ajunge să distorsioneze sau să suprimă vocea umană³². Algoritmi puternici, precum WebRTC AEC3, conțin elemente de protecție avansate care blochează (îngheață) calculul filtrelor exact în momentul detectării intervenției³².

Pentru interfețele bazate pe browser care interacționează cu nucleul Python Antigravity, este imperativă utilizarea Web Worker-urilor și AudioWorklet-urilor pentru izolarea firelor de execuție, deoarece variațiile platformelor dictează viabilitatea proiectului. Motoarele din browsere sunt de o eficiență diametral opusă: implementarea AEC din Mozilla Firefox se dovedește a fi una din cele mai precare de pe piață, pe când Safari sau hardware-ul nativ Apple operează la rate impecabile, deși pe iOS impun strategii avansate (ex. menținerea forțată a procesării WebRTC active prin „session keepalive” pe fundal)³³. Alternativa de a folosi AEC pe backend necesită un flux dublu de canale cu o cronometrare absolut perfectă între semnale, nefiind recomandată datorită dificultăților de implementare și lipsei de acces hardware²⁵.

Platformă Software / Browser	Suport Nativ AEC (Comportament Implicit)	Riscuri și Mitigări
Google Chrome (Desktop)	Robust (echoCancellation: true)	Comportament echilibrat, dar limitat de calitatea camerei acustice.
Mozilla Firefox	Foarte slab	Va lăsa vocea TTS să treacă prin microfon; necesită rutare server-side.
Safari (iOS)	Excelent (Integrare hardware)	Oprirea WebRTC în background resetează filtrele, necesitând keepalive.

Android Chrome	Variabil în funcție de producător OEM	Recomandare testare empirică; terminalele ieftine blochează implementările.
----------------	---------------------------------------	---

5.3. Arhitectura Hibridă pentru Reducerea Latenței la Întrerupere

Din momentul inițierii întreruperii vocale, așteptarea ca serverul să proceseze semnalul vocal, să identifice că utilizatorul a vorbit, să comunice înapoi clientului oprirea difuzoarelor presupune, în funcție de starea rețelei, o inerție catastrofală²⁴.

Arhitectura asistentului optim impune o abordare hibridă pentru VAD, distribuită între front-end și server-side. Un filtru VAD neural ușor este rulat direct pe mediul de client (de exemplu, convertit în WebAssembly în browser sau integrat în sistemul hardware local). În fracțiunea de secundă în care un eșantion vocal atinge pragul de validitate, clientul blochează local redarea buffer-ului TTS și transmite simultan o notificare de suprimare („truncate signal” sau „barge-in”) prin canalul bidirecțional. Backend-ul Python prinde acest eveniment (printr-un `asyncio.Event`), anulează calculul curent generat de LLM în Antigravity SDK și resetează mașinăria pentru o nouă secvență STT²⁴.

6. Model Context Protocol: Integrarea Instrumentală și Automatizarea

Un model AI izolat cognitiv nu poate schimba structura mediului. Interacțiunea cu date externe, citirea fișierelor și manipularea sistemului de operare cad în sarcina Model Context Protocol, implementat în Antigravity prin biblioteca auxiliară FastMCP³⁵.

6.1. Abstractizarea și Definirea Instrumentelor

FastMCP orchestrează tranziția complexă dintre un pachet JSON și structurile de date Python interne. Printr-un singur apel de decorator (`@mcp.tool`), platforma extrage semnătura funcției (fie ea sincronă `def` sau asincronă `async def`) și creează schema de comunicare MCP corespunzătoare, pe care o expune către Antigravity. Funcțiile asincrone rulează natural pe bucla de evenimente a sistemului (event loop), în vreme ce funcțiile sincrone, pentru a nu sufoca performanța buclei asincrone (care ar degrada direct fluxul vocal), sunt trecute automat într-un threadpool paralel³⁵. Funcțiile ce reclamă fixare obligatorie pe firul de execuție principal (ex: elemente GUI, conexiuni Windows COM) presupun un flag special `run_in_thread=False`³⁶.

Un avantaj major este rezolvarea automată și interogarea schemelor Pydantic; FastMCP simplifică structurile dincolo de notația standard JSON `$ref`, care e limitată pe anumite ecosisteme de client, integrându-le („inline”) direct în flux³⁶.

6.2. Documentarea, Validarea și Ascunderea Parametrilor

Agentul Antigravity primește un instrument MCP sub forma unui set de constrângeri logice. Pentru a direcționa deciziile agentului (de pildă, impunând un anumit domeniu de valori pentru volumul boxelor), FastMCP folosește intensiv adnotările Pydantic. Prin sintaxa bazată pe Annotated și Field, se pot defini clar limite stricte: `ge` (greater or equal), `max_length`, `tipare`

RegExp, oferind LLM-ului o descriere pe înțeles a restricțiilor (description="Target width în pixels")³⁶. Astfel, sistemul are mecanisme interne native pentru interogare coerentă, refuzând automat un input aberant de la model.

Multe integrări locale reclamă chei de acces API locale, token-uri sau adrese specifice ale mașinii host (pe care agentul nu are nevoie, sau din rațiuni de siguranță, nu trebuie să le cunoască). Mecanismul de ascundere este activat prin injecția de dependențe (Depends()). De exemplu, user_id: str = Depends(get_user_id) oferă garanția că parametrii cheie nu vor fi vizibili în logurile conversaționale generate către motorul LLM³⁶.

6.3. Interacțiunea la nivel de Sistem de Operare

Controlul hardware se obține prin integrarea fișierelor de scripting clasice ale platformei Python. Module ca subprocess (pentru executarea rutinelor native bash sau dir) și biblioteca de interfațare grafică de tip macro pyautogui permit o flexibilitate masivă. Instrumentele pot lansa programe, recunoaște iconițe și chiar face clic-uri automate, extinzând exponențial influența asistentului dincolo de barierele ferestrei CLI³⁷. Aceste acțiuni de intervenție sistem devin „uneltele” pe care agentul le poate planifica cu libertate în momentul în care asociază logica lor rezolvării unei solicitări specifice.

Tip de Anotare FastMCP	Exemple Python	Utilizare în Antigravity Tools
Valori Scalare Standard	int, str, bool	Setarea stărilor binare, trimitere de instrucțiuni simple.
Containere și Colecții	list[str], dict[str, int]	Returnare metadate din Home Assistant; listare structuri.
Restricții Pydantic	Field(..., ge=0, le=100)	Parametri de control restricționați (volum, intensitate lumini).
Dependențe invizibile	Depends(inject_credential)	Ascunderea token-ului de autorizare, logarea ID client.

7. Integrarea Senzorială și IoT prin Home Assistant

Un asistent adevărat trebuie să perceapă și să reacționeze la coordonatele fizice ale locației sale de baștină. Platforma open-source Home Assistant (HA) funcționează de facto ca panoul electric cognitiv al ecosistemelor IoT, abstractizând complexitatea protocoalelor (Zigbee, Wi-Fi, Z-Wave).

Jarvis interacționează cu panoul HA prin intermediul unor servere dedicate FastMCP sau scripturi izolate prin REST API, servite prin portul standard (implicit 8123) de pe interfața routerului local³⁸. Soluția directă este implementarea request-urilor Python via module asincrone (aiohttp).

Cererile de tip GET adresate rutei /api/states filtrează ansamblul uriaș de variabile după cheia entity_id și încarcă setul de valori descriptive (unit_of_measurement, friendly_name, last_updated). În schimb, cererile de tip POST efectuează comenzile, manipulând cheia obiectului JSON: {"state": "on"} (aprinzând luminile sau activând termostatele). Această simbioză asigură o rutină bidirecțională de feedback: agentul schimbă starea și, instantaneu, citește noile atribute de senzori emise de platforma HA, formulând vocalizarea TTS a rezultatului exact, augmentând astfel precizia operațională (ex: „Temperatura este de -3.9 °C” după citirea parametrului sensor.weather_temperature)³⁸.

8. Orchestrarea Paralelă a Sarcinilor și Gestionarea Sub-Agenților

Fluxurile de procesare vocală și raționament secvențial sunt capabile de sarcini discrete, scurte. Când asistentului i se trasează sarcini colosale sau exploratorii ce reclamă interogarea intensivă a mediului web („Analizează platformele de tehnologie și redactează un sumar pentru top 10 știri AI în fundal”³), firul principal de ascultare a utilizatorului riscă să înghețe, compromițând total arhitectura vocala asincronă.

Pentru depășirea acestei asimetrii operaționale, Google Antigravity folosește un tip de delegare complex prin clasa ParallelAgent. Sub-agenții paraleli acționează ca filiale izolate ale sistemului, care rulează ramuri distincte (independent branches) în fundal, primind fiecare un InvocationContext partajat sau propriu⁴⁰.

În abordarea arhitecturală Jarvis, se utilizează instanțe de baze de date in-memory (ex. InMemorySessionService) și chei specifice cu timestamp-uri pe post de jurnale izolate, de tipul sub_task_answers. Sub-agentul, în ciuda faptului că împarte același LLM general, primește acces restricționat la instrumente (Toolsets), iar în timp ce rezolvă operațiuni masive sub umbrela metodelor de clasă suprascrise _run_async_impl(self, ctx), fluxul asistentului vocal principal (ce monitorizează FastMCP și Whisper) nu este perturbat sub nicio formă⁴⁰.

Finalizarea sub-taskului trimite un eveniment (event type) înapoi, care se suprapune contextului global; LLM-ul integrând, finalmente, și rezultatul procesat în conversația curentă. Aceasta rezolvă eleganța multitasking-ului, asistentul primind capacități veritabile de paralelism.

9. Planul de Execuție (Blueprint) pentru Google Antigravity

Următoarea secțiune reprezintă esența generativă a cercetării. Acest plan strategic (blueprint) conține un pachet de instrucțiuni tehnice detaliate care translează complet toată argumentația arhitecturală într-un format digerabil pentru un agent AI.

Instrucțiunile trebuie copiate și furnizate direct în interfața de promptare (IDE sau CLI) a Antigravity, delegând asistentului nativ responsabilitatea de a autogenera platforma „Jarvis”.

Specificația de Proiect pentru Construirea Arhitecturii "Jarvis"

Titlu Proiect: Jarvis - Asistent Autonom Multimodal în Cascadă cu Control FastMCP și IoT (Home Assistant)

Rolul Tău (Antigravity AI): Ești Arhitectul Principal de Sistem. Obiectivul tău este să generezi baza de cod (Python 3.12+), cu fișiere configurate complet, pentru un agent vocal complet autonom. Sistemul folosește Google Antigravity SDK pentru raționament, faster-whisper + silero-vad pentru STT, Kokoro-82M pentru TTS asincron, suportă funcții de barge-in (prin anularea redării pe client) și interacționează prin servere FastMCP externe cu API-uri REST (Home Assistant).

Structura Logică și Etapele de Implementare:

- **ETAPA 1: Inițializarea și Fundamentul SDK-ului Antigravity**
 - Generarea structurii directe: app/core/, app/audio/, app/tools/, și app/memory/.
 - Generarea fișierului requirements.txt cu versiunile corecte pentru modulele: google-antigravity, fastmcp, pydantic, faster-whisper, silero-vad, sounddevice, aiohttp, onnxruntime. (Asigură-te de clarificarea instrucțiunilor de instalare a pachetelor de compilare CTranslate2).
 - În core/main.py: Creează bucla async def main(), cu o configurație de clasă LocalAgentConfig() setată cu un prompt generalist în fișier separat, permițând toate capacitățile locale (CapabilitiesConfig()). Aceasta trebuie să instanțieze managerul de context async with Agent(config) as agent:.
- **ETAPA 2: Subsistemul Audio (Pipeline în Cascadă și AEC logic)**
 - În audio/stt.py: Dezvoltă o clasă pentru captarea audio prin sounddevice pe un fir paralel. Implementează logica mașinii de stări cu modelul VAD Silero (prag critic minim de tăcere de 500 ms). Buffer-ele obținute se expediază modelului WhisperModel('large-v3-turbo', compute_type='int8'). Funcția va returna un stream asincron al transcriptului vocal (generator).
 - În audio/tts.py: Proiectează legăturile necesare cu fișierul nativ ONNX pentru modelul Kokoro-82M (la frecvența 24kHz). Construiește o funcție asincronă ce citește token-urile text direct de la iterarea obiectului ChatResponse (din Antigravity) și inițiază emiterea pachetelor audio imediat.
 - **Managementul Întreruperilor (Barge-in):** Integrează un semnal global de notificare (asyncio.Event). În cazul în care modulul VAD percepe sunet valid uman în timp ce audio/tts.py rulează, forțează eliberarea (flush) redării audio (AEC software logic hook) și anulează procesarea fluxului LLM, readucând starea aplicației la faza de ascultare pură.
- **ETAPA 3: Persistența și Integrarea MCP (Tools & IoT)**
 - În tools/mcp_server.py: Construiește instanța mcp = FastMCP("JarvisControls"). Utilizează Pydantic (Annotated și Field) pentru descrierea strictă a metodelor. Definirea unui tool specific prin decoratorul @mcp.tool care utilizează aiohttp pentru a efectua cereri REST GET/POST spre IP-ul localhost:8123/api/states. Permite manipularea dispozitivelor și prelucrarea informațiilor. Adaugă o funcție auxiliară care apelează module tip subprocess sau automatizări via OS.
 - În memory/session.py: Asigură scriptul de pornire automată care redactează o etichetă

(RECAP timestamped) în sesiunea din logurile fișierului JARVIS_MEMORY.md local, prelucrând tranzițiile cu și fără date anterioare. Configurația va trimite o rutină de rechemare înainte de un răspuns legat de trecut, dacă integrează logica SurrealDB MCP.

Mecanism de Dialog:

Analizează cu atenție specificația de mai sus. Extrage un fișier tip implementation_plan.md cuprinzând sarcinile tale de dezvoltare segmentate logic. Nu genera text inutil. Așteaptă validarea mea pentru a începe generarea fizică a modulelor, începând cu zona cea mai critică a latenței: Subsistemul Audio și Barge-in din ETAPA 2.

10. Concluzii

Trecerea de la concepte asistențiale abstracte la implementarea arhitecturală fizică a unei entități autonome, controlate direct la nivel de sistem de operare, este esențial dependentă de convergența tehnologiilor din sfera AI pe terminal local (edge AI). Prin instrumentarea pachetului SDK Google Antigravity în sinergie absolută cu interfețele FastMCP, s-a obținut abstractizarea completă a mecanismelor de control asupra funcțiilor complexe. Validarea incontestabilă a modelului cu arhitectură în cascadă față de alternativele orientate direct (S2S), cuplate cu capabilități de reducere a găturilor tehnice (precum Whisper cuantizat INT8 și rutine VAD neurale riguroase ce respectă cadrele audio minimale) constituie pilonul unei integrări impecabile cu sistemele fizice și senzorii asociați prin Home Assistant. Eliminarea erorilor de „double-talk” prin suprimarea VAD descentralizată transformă, definitiv, Jarvis dintr-un simplu program recitativ într-o prezență tehnologică interactivă asincronă, pregătită pentru fluxuri decizionale robuste în viața de zi cu zi. Executarea directivelor de la nivelul final asigură o rată de succes imediată în generarea mediului de dezvoltare automatizat.

Lucrări citate

1. Introducing Google Antigravity, a New Era in AI-Assisted Software, <https://antigravity.google/blog/introducing-google-antigravity>
2. Build with Google Antigravity, our new agentic development platform, <https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>
3. Antigravity agent | Gemini API - Google AI for Developers, <https://ai.google.dev/gemini-api/docs/antigravity-agent>
4. Home | Google Antigravity Docs, <https://antigravity.google/docs/home/>
5. Download - Google Antigravity, <https://antigravity.google/download>
6. How to Use Google's Antigravity CLI for AI Code Assistance, <https://realpython.com/antigravity-cli/>
7. Tutorial | Google Antigravity Docs, <https://antigravity.google/docs/cli/tutorial/>
8. GitHub - google-antigravity/antigravity-sdk-python: A Python library, <https://github.com/google-antigravity/antigravity-sdk-python>
9. Antigravity SDK, <https://antigravity.google/product/antigravity-sdk>
10. Introducing Antigravity SDK: Transforming Development with AI, <https://allen.hutchison.org/2026/05/19/agents-as-building-blocks/>
11. Google Antigravity SDK,

- <https://antigravity.google/blog/introducing-google-antigravity-sdk>
12. How I Built a Persistent Memory System for Antigravity (with /start, <https://discuss.ai.google.dev/t/how-i-built-a-persistent-memory-system-for-antigravity-with-start-workflow-session-management-lazygravity/128640>)
 13. Antigravity | Coding assistants | Agent Memory - SurrealDB, <https://surrealdb.com/docs/agent-memory/integrations/mcp-server/coding-assistants/antigravity>
 14. Can Speech LLMs Think while Listening? - arXiv, <https://arxiv.org/html/2510.07497v1>
 15. Cascaded Voice Agents vs Speech-to-Speech - Gadium, <https://gadium.ai/content/cascaded-voice-agent-vs-speech-to-speech-2026>
 16. Cascaded vs Speech-to-Speech Voice Architecture - Inworld AI, <https://inworld.ai/resources/cascaded-vs-speech-to-speech-voice-architecture>
 17. Speech-to-Speech vs Cascaded Voice AI: Which Architecture, <https://www.coval.ai/blog/speech-to-speech-vs-cascaded-voice-ai-which-architecture-should-you-deploy/>
 18. Speech-to-Speech vs Cascade: Voice Agent Architecture - Deepgram, <https://deepgram.com/learn/speech-to-speech-vs-cascade-voice-agent-architecture>
 19. A Reproducible and Verifiable Framework for Evaluating Tool ... - arXiv, <https://arxiv.org/html/2605.15104v1>
 20. Whisper WebUI with a VAD for more accurate non-English ... - GitHub, <https://github.com/openai/whisper/discussions/397>
 21. Real-time pipeline for YouTube Audio -> Whisper -> LLM -> SSE, https://www.reddit.com/r/MachineLearning/comments/1th713c/architecture_advice_realtime_pipeline_for_youtube/
 22. Self-Host Faster-Whisper on GPU Cloud: Production Deployment, <https://www.spheron.network/blog/faster-whisper-gpu-cloud-production-deployment-guide/>
 23. Voxwire: Designing a Privacy-Preserving, Fully Offline Speech, <https://xiv.jst.go.jp/index.php/xiv/preprint/download/3482/8240/7661>
 24. The Art of Interruption: VAD Strategies for Fluid AI Conversations, https://dev.to/deepak_mishra_35863517037/the-art-of-interruption-vad-strategies-for-fluid-ai-conversations-15bh
 25. Audio Preprocessing & Barge-In - Deepgram's Docs, <https://developers.deepgram.com/guides/deep-dives/audio-preprocessing-charge-in>
 26. 2025 Voice AI Guide How to Make Your Own Real-Time ... - Medium, <https://medium.com/@programmerraja/2025-voice-ai-guide-how-to-make-your-own-real-time-voice-agent-part-3-7ca328aaea72>
 27. pykokoro - PyPI, <https://pypi.org/project/pykokoro/>
 28. This Tiny 82M Model Just Beat Most TTS APIs (Runs Locally), <https://www.youtube.com/watch?v=bdf6BxyxCnQ>
 29. Deploy Open-Source TTS on GPU Cloud: Kokoro, Fish Speech, and, <https://www.spheron.network/blog/deploy-open-source-tts-gpu-cloud-2026/>

30. Building a Fully Local AI Voice Receptionist. Looking for architecture, https://www.reddit.com/r/AIReceptionists/comments/1usg72h/building_a_fully_local_ai_voice_receptionist/
31. Voice Agent Platform Architecture: The Stack Behind Sub-300ms, <https://www.channel.tel/blog/voice-agent-platform-architecture-sub-300ms-responses>
32. Barge-in and interruption handling for on-device voice agents, <https://www.runedge.ai/blog/barge-in-interruption-handling-on-device-voice>
33. Voice AI Echo Cancellation: Causes, Fixes, and Best Practices - Coval, <https://www.coval.ai/blog/voice-ai-echo-cancellation/>
34. Frontend Architecture of a Voice Agent Interface - Medium, <https://medium.com/@ujjwaltiwari2/frontend-architecture-of-a-voice-agent-interface-6236bfc393ba>
35. FastMCP: The Framework for MCP - FastMCP, <https://gofastmcp.com/getting-started/welcome>
36. Tools - FastMCP, <https://gofastmcp.com/servers/tools>
37. GitHub - digithree/automac-mcp: An experimental MCP server that, <https://github.com/digithree/automac-mcp>
38. REST API - Home Assistant Developer Docs, <https://developers.home-assistant.io/docs/api/rest/>
39. ha-mcp - PyPI, <https://pypi.org/project/ha-mcp/>
40. Subagents | Google Antigravity Docs, <https://antigravity.google/docs/sdk/subagents/>
41. Parallel workflow - Agent Development Kit (ADK), <https://adk.dev/agents/workflow-agents/parallel-agents/>
42. Multiple Agents run in parallel with different inputs. How to implement?, <https://github.com/google/adk-python/issues/2124>